

Problem A. Ada no País das Maravilhas

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Ada decidiu tirar férias e fazer uma viagem inesquecível para o País das Maravilhas! Chegando lá, ela percebeu que precisaria converter todo o seu dinheiro para a moeda local para poder aproveitar sua estadia.

Ela foi até uma casa de câmbio e descobriu que 1 moeda do país no qual ela mora equivale a exatamente Y moedas no sistema monetário do País das Maravilhas.



Sabendo que Ada levou X moedas do seu país em sua bagagem, sua tarefa é ajudá-la a calcular quantas moedas ela terá para gastar no País das Maravilhas após realizar a troca.

Input

A entrada consiste em duas linhas com um inteiro em cada.

A primeira linha contém o inteiro X ($1 \leq X \leq 1000$), que representa a quantidade de moedas que Ada levou em sua viagem.

A segunda linha contém o inteiro Y ($1 \leq Y \leq 1000$), que é o valor correspondente à taxa de conversão da moeda (quantas moedas do País das Maravilhas equivalem a 1 moeda do país de Ada).

Output

A saída deve conter um único número inteiro, que representa o valor total, em moedas, que Ada possui de acordo com o sistema monetário do País das Maravilhas.

Examples

standard input	standard output
1 1	1
3 5	15
10 20	200

Problem B. Batalha Lexicográfica

Input file: standard input
Output file: standard output
Time limit: 3 seconds
Memory limit: 256 megabytes

Ada e Py reencontram-se depois de muito tempo e decidem disputar um jogo, o qual será intermediado por um juiz.

No início do jogo, Ada possui uma string não vazia s_A , enquanto Py tem uma string s_P , a qual também é não vazia. O jogo consiste em uma sequência de Q operações, que devem ser processadas em ordem. Tais operações podem ser de 3 tipos diferentes (1, 2 ou 3), descritas a seguir:

1. Dados um inteiro k e uma string s , adicione k cópias da string s ao final da string s_A . Em outras palavras, após uma operação do tipo (1 k s), a string s_A passa a ser igual à concatenação de s_A com k cópias da string s ;
2. Dados um inteiro k e uma string s , adicione k cópias da string s ao final da string s_P . Em outras palavras, após uma operação do tipo (2 k s), a string s_P passa a ser igual à concatenação de s_P com k cópias da string s ;
3. Indica uma rodada do jogo. Em cada rodada, Ada e Py são solicitadas a enviar suas strings (s_A e s_P , respectivamente) ao juiz. Antes de enviá-las, tanto Ada quanto Py podem reordenar suas strings da maneira que bem entenderem. Em outras palavras, Ada pode enviar uma permutação qualquer de s_A , enquanto Py pode enviar uma permutação qualquer da string s_P . Ao receber as duas strings, o juiz declara como vencedora da rodada aquela que tiver enviado a string lexicograficamente menor. Se ambas as strings forem iguais, o juiz declara um empate como resultado da rodada.

É garantido que s_A e s_P são inicialmente formadas apenas por letras latinas minúsculas, assim como todas as strings s presentes nas operações dos tipos 1 e 2.

Dizemos que uma string A é lexicograficamente menor que uma string B se, e somente se, uma das duas condições a seguir for satisfeita:

1. A é um prefixo de B e A é diferente de B ;
2. Na primeira posição em que A e B diferem, A apresenta uma letra que antecede a letra correspondente em B na ordem alfabética.



Sua tarefa é, para cada operação do tipo 3, informar qual será o resultado determinado pelo juiz, considerando que Ada e Py jogam de maneira ótima (levando em conta as strings s_A e s_P no momento em que a rodada ocorre).

Input

A primeira linha da entrada contém a string não vazia s_A no início do jogo.

A segunda linha contém a string não vazia s_P no início do jogo.

A terceira linha contém um inteiro Q ($1 \leq Q \leq 10^5$), indicando o número de operações que irão ocorrer ao longo do jogo.

As próximas Q linhas começam com um inteiro t ($1 \leq t \leq 3$), indicando qual o tipo daquela operação. Se $t = 1$ ou $t = 2$, t é sucedido por um inteiro k ($1 \leq k \leq 10^5$) e por uma string não vazia s .

É garantido que todas as strings da entrada contêm apenas letras latinas minúsculas e que o somatório dos comprimentos de todas elas não excede $2 \cdot 10^5$.

Output

Para cada operação do tipo 3, imprima **ADA**, caso Ada seja a vencedora da rodada; **PY**, caso Py seja a vencedora da rodada ou **EMPATE**, caso o juiz determine que a rodada em questão empatou.

Examples

standard input	standard output
mfp mfp 7 3 2 1 s 3 2 1 f 3 1 2 fps 3	EMPATE ADA PY ADA
ada py 5 1 100000 ada 1 100000 py 1 100000 ada 2 1 aba 3	ADA

Problem C. Cortando Marca-Páginas

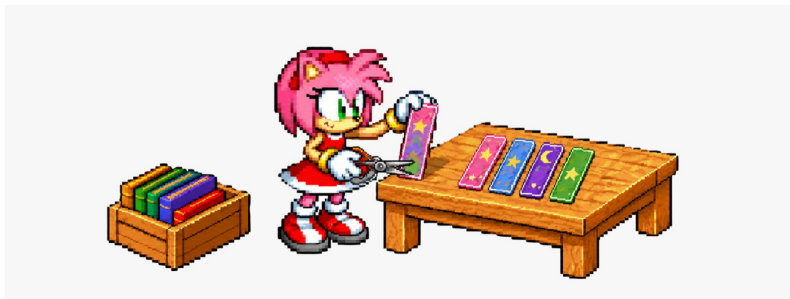
Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Amy é dona de uma pequena papelaria no centro da cidade. Recentemente, ela desenvolveu um novo modelo de marcador de página muito especial que está fazendo sucesso entre os estudantes.

Esses marcadores são feitos de um material resistente e colorido, cortados de uma grande folha retangular que Amy compra de seu fornecedor. Cada folha retangular tem exatamente X centímetros de largura por Y centímetros de comprimento.

Para otimizar sua produção e evitar desperdício, Amy corta a folha em marcadores de página com exatamente 1 centímetro de largura por 7 centímetros de comprimento. Por questões técnicas da máquina de corte que Amy utiliza, ela deve escolher apenas uma orientação para todos os marcadores em cada folha — ou todos alinhados horizontalmente, ou todos alinhados verticalmente com as bordas da folha.

No fim do dia, Amy precisa saber quantos marcadores completos ela consegue produzir a partir de cada folha retangular, considerando a melhor orientação possível. Ela não considera pedaços incompletos, pois não podem ser vendidos.



Como Amy está muito ocupada atendendo clientes, ela pediu sua ajuda para criar um programa que calcule rapidamente o número máximo de marcadores que podem ser produzidos a partir de uma folha retangular com dimensões X por Y , escolhendo a orientação que maximiza a produção.

Input

A entrada consiste em uma única linha contendo dois números inteiros X e Y ($1 \leq X, Y \leq 1000$), representando, respectivamente, a largura e o comprimento da folha retangular em centímetros.

Output

Imprima um único número inteiro representando o número máximo de marcadores de página completos ($1 \text{ cm} \times 7 \text{ cm}$) que podem ser produzidos.

Examples

standard input	standard output
1 10	1
700 100	10000

Problem D. Dividindo Figurinhas

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Ada é uma grande fã de futebol e está muito animada completando o seu álbum de figurinhas da Copa do Mundo. Após comprar e abrir centenas de pacotinhos, ela finalmente conseguiu completar o álbum, mas acabou acumulando várias figurinhas repetidas.

Para organizar sua coleção, Ada separou suas figurinhas repetidas por país. Como há N seleções de diferentes países participando da Copa, ela fez N pilhas, de modo que cada pilha contém apenas figurinhas de uma mesma seleção. A i -ésima pilha possui exatamente A_i figurinhas.

Não vendo sentido em acumular figurinhas repetidas, Ada decidiu distribuí-las entre seus amigos. Com o intuito de manter a organização e a justiça na divisão, as seguintes condições deverão ser obedecidas:

- Cada amigo contemplado deve receber figurinhas de uma única seleção (ou seja, figurinhas que estão em uma mesma pilha);
- Todos os amigos contemplados devem receber a mesma quantidade de figurinhas;
- Todas as figurinhas de Ada devem ser distribuídas (isto é, nenhuma figurinha deve permanecer com Ada).



Ada quer que cada amigo receba a maior quantidade de figurinhas possível. Consequentemente, isso significa que ela precisa escolher o menor número de amigos possível para receber a doação, respeitando todas as regras acima.

Sua tarefa é ajudar Ada a calcular qual é o número mínimo de amigos que poderão ser contemplados com suas figurinhas repetidas.

Input

A entrada consiste em duas linhas.

A primeira linha contém um inteiro N ($1 \leq N \leq 10^5$), representando o número de países participando da Copa (e, consequentemente, o número de pilhas de figurinhas repetidas de Ada).

A segunda linha contém N inteiros separados por espaço $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$), em que A_i representa a quantidade de figurinhas repetidas do i -ésimo país.

Output

A saída deve conter um único inteiro: o número mínimo de amigos que podem ser escolhidos para receber as figurinhas de Ada, obedecendo às condições descritas.

Examples

standard input	standard output
1 10	1
3 10 15 7	32
4 24 60 36 48	14

Note

No primeiro caso de teste, Ada possui figurinhas repetidas de um único país, totalizando 10 figurinhas. Ela pode entregar as 10 figurinhas para um único amigo de acordo com as regras estabelecidas.

No segundo caso de teste, Ada possui pilhas com 10, 15 e 7 figurinhas repetidas. A única divisão que respeita as condições impostas consiste em entregar 1 figurinha para cada amigo. Sendo assim, ela precisará presentear $10 + 15 + 7 = 32$ amigos.

Problem E. Estratégia de Pneus

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

A temporada de 2026 da MFP Racing, uma das competições de automobilismo mais importantes do mundo, está prestes a começar. Você foi contratada como consultora de uma das equipes e sua primeira tarefa é desenvolver um programa que crie uma estratégia de pneus para a corrida de abertura, minimizando o tempo total da prova.

Existem três tipos de pneus disponíveis para as pilotas:

- Macio — rápido, mas dura menos voltas;
- Médio — equilibrado;
- Duro — mais lento, porém apresenta maior durabilidade.

Cada tipo de pneu possui um tempo específico gasto por volta e uma durabilidade máxima. Além disso, cada corrida possui N voltas. O carro deve completar todas elas, sendo permitido largar com qualquer tipo de pneu.

Durante a corrida, podem ser feitas trocas de pneus (pit-stops) nos boxes. Essas paradas ocorrem sempre após a conclusão de uma volta e não são contabilizadas como voltas da corrida (ou seja, fazer um pit-stop não reduz o número de voltas que ainda precisam ser dadas). Graças à grande eficiência dos mecânicos da equipe, você já sabe que cada parada adiciona um tempo fixo P , em segundos, ao tempo total da prova. Contudo, existe um limite de, no máximo, S pit-stops por corrida.



Sua tarefa é calcular o tempo mínimo necessário para finalizar a corrida, respeitando o limite de trocas e as características de cada pneu. É garantido que sempre existe ao menos uma estratégia com a qual é possível concluir a prova.

Input

A entrada consiste em três linhas.

A primeira linha contém três inteiros N , P e S ($2 \leq N \leq 1000$, $1 \leq P \leq 100$, $0 \leq S \leq 5$), representando, respectivamente, o número de voltas, o tempo (em segundos) gasto em cada troca de pneus e o número máximo de trocas permitidas.

A segunda linha contém três inteiros t_1 , t_2 e t_3 ($1 \leq t_1 < t_2 < t_3 \leq 300$), representando o tempo gasto por volta (em segundos) de cada tipo de pneu (macio, médio e duro, respectivamente).

A terceira linha contém três inteiros d_1 , d_2 e d_3 ($1 \leq d_1 < d_2 < d_3 \leq N$), que representam o número máximo de voltas que cada tipo de pneu (macio, médio e duro, respectivamente) suporta antes de ser necessária a realização de uma troca nos boxes.

Output

Imprima um único número inteiro: o **menor tempo total possível**, em segundos, para completar a corrida.

Example

standard input	standard output
10 20 2 80 83 90 4 6 7	838

Note

A melhor estratégia para o caso do exemplo consiste em:

- 4 voltas com pneu macio ($4 \times 80 = 320$ s)
- pit-stop (20 s)
- 6 voltas com pneu médio ($6 \times 83 = 498$ s)
- total = $320 + 20 + 498 = \mathbf{838}$ segundos.

Problem F. Fuga da Masmorra

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Ada é uma ave que adora se reunir com seus amigos da floresta para jogar RPG, um jogo no qual eles criam histórias juntos e usam dados para decidir o sucesso de suas ações. Na partida de hoje, a personagem de Ada encontra-se em uma grande enrascada: ao explorar uma antiga masmorra subterrânea, ela ativou uma armadilha e agora o teto está descendo rapidamente sobre o seu grupo.

Para conseguir escapar dessa situação, o Mestre, amigo que organiza e dita as regras do jogo, avisa que Ada precisa passar por um teste de sorte. O teste em questão consiste em rolar N dados diferentes ao mesmo tempo. O i -ésimo dado possui d_i faces, de modo que, ao rolá-lo, Ada pode obter, com a mesma probabilidade, qualquer inteiro no intervalo $[1, d_i]$.

A regra imposta pelo Mestre é simples: o grupo só conseguirá se salvar se o maior valor obtido, considerando todos os N dados lançados, for maior ou igual a um valor K .



Ada gostaria de saber suas reais chances de sobrevivência antes de rolar os dados. Sabendo a quantidade de dados, o número de faces de cada um e o valor K estipulado pelo Mestre, sua tarefa é calcular a probabilidade de Ada e seu grupo conseguirem escapar da armadilha.

Input

A entrada consiste em duas linhas.

A primeira linha contém dois inteiros N, K ($1 \leq N \leq K \leq 100$), separados por espaço, que representam, respectivamente, a quantidade de dados que Ada deve rolar e o valor estabelecido pelo Mestre para o teste.

A segunda linha contém N inteiros d_i separados por espaço, ($2 \leq d_i \leq 100$), que representam a quantidade de lados de cada um dos N dados.

É garantido que $K \leq d_i$ para algum $1 \leq i \leq N$.

Output

A saída deve conter um único número real r tal que $0 \leq r \leq 1$, que representa a probabilidade de Ada conseguir passar no teste.

Sua resposta será considerada correta se a diferença relativa ou absoluta para a resposta do juiz não ultrapassar 10^{-6} .

Dica: Para imprimir um número de ponto flutuante r com precisão em Python, você pode usar:

```
print(f"{r:.10f}")
```

Em C++, você pode usar o `setprecision`:

```
std::cout << std::fixed << std::setprecision(10) << r << "\n";
```

Examples

standard input	standard output
1 2 2	0.5000000000000000
1 5 5	0.2000000000000000
2 3 3 3	0.5555555555555556
3 4 5 2 3	0.4000000000000000

Problem G. Ganho ou Perco?

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

Ada é uma aluna fascinada por jogos de estratégia e foi desafiada por seu professor de física para uma partida de um jogo numérico.

O jogo começa com um número inteiro positivo N escrito na lousa. Os dois jogadores alternam turnos e, por ser a desafiada, Ada tem o direito de fazer o primeiro movimento. Em cada turno, o jogador da vez deve escolher um número que seja uma potência de 2 (ou seja, 1, 2, 4, 8, 16, ...) e subtrair esse valor do número escrito na lousa, apagando o número antigo e escrevendo o novo resultado. Só é permitido subtrair um valor que seja menor ou igual ao número atual na lousa.

Formalmente, se o número escrito na lousa é x , o jogador da vez pode escolher um inteiro $k \geq 0$ tal que $2^k \leq x$ e substituir x por $x - 2^k$.

Vence o jogador que fizer o movimento que reduz o número na lousa a exatamente 0.



Ada é extremamente competitiva e odeia perder. Antes de aceitar o desafio do professor, ela quer analisar o número N inicial e descobrir se ela tem a garantia de vitória ou se a derrota é inevitável (assumindo que tanto ela quanto o professor jogarão de forma ótima).

Escreva um programa que, dado o número N escrito na lousa, responda à pergunta que não sai da cabeça de Ada: “Ganho ou Perco?”.

Input

A única linha da entrada contém apenas um inteiro N ($1 \leq N \leq 10^{18}$), indicando qual o número escrito na lousa no início do jogo.

Output

Imprima **GANHA** se Ada possuir uma estratégia vencedora jogando primeiro. Caso contrário (ou seja, caso o professor possua a estratégia vencedora sendo o segundo a jogar), imprima **PERDE**.

Examples

standard input	standard output
2	GANHA
30	PERDE
10000000000000000	GANHA

Problem H. Hora de Caminhar

Input file: standard input
Output file: standard output
Time limit: 2 seconds
Memory limit: 512 megabytes

A fase final da Maratona Feminina de Programação (MFP) ocorrerá novamente na Unicamp em 2026. Para receber as finalistas da melhor forma possível, a organização preparou um mapa destacando N pontos de interesse cruciais no campus (como o Instituto de Computação e o Restaurante Universitário).

Para garantir que nenhuma participante se perca, a organização mapeou exatamente $N - 1$ caminhos principais conectando esses locais, formando uma estrutura na qual existe uma única rota entre qualquer par de pontos. Cada caminho possui um tempo estimado de caminhada em minutos.

No entanto, o campus da Unicamp é um ambiente muito dinâmico. O tempo de percurso de um caminho pode mudar repentinamente durante o dia devido a imprevistos como poças d'água após chuvas rápidas, filas gigantescas que se formam e dobram a esquina perto dos restaurantes, ou até mesmo por uma família de capivaras que decidiu tirar uma soneca no meio da calçada!

Para garantir o bem-estar das finalistas, a coordenação da MFP deseja monitorar o “Fator de Cansaço” do campus, que é definido como a soma dos tempos de caminhada, em minutos, entre todos os possíveis pares de pontos de interesse. A cada vez que ocorre um imprevisto e o tempo de um caminho é alterado, a coordenação precisa que esse fator seja imediatamente recalculado.



Dado o mapa preparado pela organização da MFP, sua tarefa é processar Q atualizações nos tempos dos caminhos ao longo do dia. Após cada mudança, você deve calcular e imprimir o Fator de Cansaço atualizado. Como as participantes vão andar bastante e a soma dos tempos pode ser muito grande, imprima os resultados em módulo $10^9 + 7$.

Input

A primeira linha de entrada contém dois inteiros N, Q ($2 \leq N \leq 2 \cdot 10^5$, $1 \leq Q \leq 2 \cdot 10^5$), que representam, respectivamente, o número de pontos de interesse no campus e o número de atualizações nos tempos dos caminhos ao longo do dia. Os pontos de interesse são numerados como $1, 2, \dots, N$.

Em seguida, há $N - 1$ linhas descrevendo os caminhos mapeados pela organização. A i -ésima dessas linhas contém três inteiros u, v e w ($1 \leq u, v \leq N$ e $1 \leq w \leq 10^9$), indicando que há um caminho entre os pontos u e v com tempo inicial de caminhada igual a w minutos. É garantido que os caminhos formam uma estrutura na qual existe exatamente uma rota entre qualquer par de pontos (uma árvore).

Por fim, há Q linhas descrevendo as atualizações decorrentes dos imprevistos. Cada linha contém dois inteiros j e x ($1 \leq j \leq N - 1$ e $1 \leq x \leq 10^9$), indicando que o tempo de caminhada do j -ésimo caminho (na ordem em que foram dados na entrada, começando do índice 1) é alterado para x minutos.

Output

Imprima Q inteiros: o “Fator de Cansaço” atualizado (a soma dos tempos de caminhada entre todos os pares de pontos de interesse), calculado em módulo $10^9 + 7$ após o processamento de cada atualização, imprimindo um por linha.

Um valor em módulo $10^9 + 7$ é igual ao resto da divisão do valor original por $10^9 + 7$.

Example

standard input	standard output
5 4	72
1 2 1	140
1 3 2	128
2 4 3	56
2 5 4	
1 6	
2 19	
4 1	
2 1	

Note

A imagem à esquerda representa a configuração inicial dos caminhos. A imagem à direita ilustra a estrutura do campus após o primeiro imprevisto, que altera o tempo de caminhada entre os pontos 1 e 2 de 1 para 6 minutos.

